

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Анализ сложности алгоритмов

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Целью работы является изучение методов и получение практических навыков анализа сложности алгоритмов.

2 Вариант задания

2.1 Задание

Написать программу, которая выполняет задание по варианту из таблицы 1. Для заполнения матрицы (там, где это нужно) можно использовать либо автоматическую генерацию данных, либо считывать заранее подготовленные данные из текстового файла. Каждое отдельное действие (заполнение данными, вывод на экран матрицы, вывод на экран результата, выполнение действия и т.п.) должно быть оформлено как функция. Программа должна содержать меню, позволяющее выбрать любое действие. Повторяющиеся действия (например, заполнение данными первой матрицы, заполнение данными второй матрицы) следует выполнять через вызов одной и той же функции с разными переменными. Размер всех матриц – произвольный, $n \times m$. В начале работы программы следует запрашивать у пользователя значения n и m . Для ввода/вывода используйте функции `cin/cout`. Разрешается доопределять задания. Все дополнительные ограничения и особенности реализации должны быть отражены в отчете в разделе «Выводы по работе».

По тексту готовой программы рассчитать теоретические временную и пространственную сложности для каждой функции и всей программы

2.2 Вариант

Вариант 11: вычислить определитель матрицы

3 Листинг программы, реализующей алгоритм

```
#include <iostream>
#include <vector>
#include <cstdlib>    // для rand() и srand()
#include <ctime>      // для time()
#include <cmath>      // для round()
using namespace std;

int n, m; // размеры матрицы: n – строки, m – столбцы

// Функция запроса размеров матрицы у пользователя
void inputSize() {
    cout << "Enter the number of rows n: ";
    cin >> n;

    // Проверка корректности ввода
    while (n <= 0) {
        cout << "n must be greater than 0. Enter again: ";
        cin >> n;
    }

    cout << "Enter the number of columns m: ";
    cin >> m;

    // Проверка корректности ввода
    while (m <= 0) {
        cout << "m must be greater than 0. Enter again: ";
        cin >> m;
    }
}

// Функция ручного ввода элементов матрицы
void fillMatrix(vector<vector<int>>& A) {
    cout << "Enter the elements of the matrix " << n << "x" << m << ":\n";

    // Проходим по всем строкам и столбцам
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << "A[" << i + 1 << "][" << j + 1 << "] = ";
            cin >> A[i][j];
        }
    }
}

// Функция заполнения матрицы случайными числами
// Числа генерируются в диапазоне [-9, 9]
void fillMatrixRandom(vector<vector<int>>& A) {
```

```

// Инициализация генератора случайных чисел
srand((unsigned int)time(0));

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {

        // rand()%19 дает числа 0..18
        // -9 сдвигает диапазон до -9..9
        A[i][j] = rand() % 19 - 9;
    }
}

cout << "Matrix filled with random numbers.\n";
}

// Функция вывода матрицы на экран
void printMatrix(const vector<vector<int>>& A) {

    cout << "\nMatrix " << n << "x" << m << ":\n";

    // Вывод элементов построчно
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {

            cout << A[i][j];

            // Табуляция между элементами для красивого вывода
            if (j < m - 1)
                cout << "\t";
        }

        cout << "\n";
    }
}

// Функция вычисления определителя методом Гаусса
// Работает только для квадратных матриц (n == m)
int calcDeterminant(const vector<vector<int>>& A) {

    // Создаем копию матрицы в типе double
    // Это нужно для точных вычислений при делении
    vector<vector<double>> B(n, vector<double>(n));

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            B[i][j] = A[i][j];
}

```

```

double det = 1.0;

// Основной цикл метода Гаусса
for (int col = 0; col < n; col++) {

    // Поиск ненулевого опорного элемента (pivot)
    int pivotRow = -1;

    for (int row = col; row < n; row++) {
        if (B[row][col] != 0) {
            pivotRow = row;
            break;
        }
    }

    // Если весь столбец нулевой – определитель равен 0
    if (pivotRow == -1)
        return 0;

    // Если опорная строка не совпадает с текущей – меняем строки
    if (pivotRow != col) {
        swap(B[pivotRow], B[col]);

        // При перестановке строк знак определителя меняется
        det *= -1;
    }

    // Умножаем определитель на опорный элемент
    det *= B[col][col];

    // Обнуляем элементы ниже опорного
    for (int row = col + 1; row < n; row++) {

        double factor = B[row][col] / B[col][col];

        for (int k = col; k < n; k++) {
            B[row][k] -= factor * B[col][k];
        }
    }
}

// Округляем результат, так как исходная матрица целочисленная
return (int)round(det);
}

// Функция вывода результата вычисления
void printResult(int det) {
    cout << "\nMatrix determinant = " << det << "\n";
}

```

```

}

// Функция отображения меню программы
void showMenu() {

    cout << "\n=== MENU ===\n";
    cout << "1. Enter matrix manually\n";
    cout << "2. Fill matrix with random numbers\n";
    cout << "3. Display matrix\n";
    cout << "4. Calculate determinant\n";
    cout << "0. Exit\n";
    cout << "Choose an option: ";
}

int main() {

    // Запрос размеров матрицы
    inputSize();

    // Создание матрицы n x m
    vector<vector<int>> A(n, vector<int>(m, 0));

    bool filled = false; // флаг: заполнена ли матрица
    int choice;

    // Основной цикл программы
    do {

        showMenu();
        cin >> choice;

        switch (choice) {

            // Ручной ввод матрицы
            case 1:
                fillMatrix(A);
                filled = true;
                cout << "Matrix entered.\n";
                break;

            // Заполнение случайными числами
            case 2:
                fillMatrixRandom(A);
                filled = true;
                break;

            // Вывод матрицы
            case 3:

```

```

        if (!filled) {
            cout << "First fill the matrix (option 1 or 2).\n";
        } else {
            printMatrix(A);
        }
        break;

// Вычисление определителя
case 4:
    if (!filled) {
        cout << "First fill the matrix (option 1 or 2).\n";
    }
    else if (n != m) {
        cout << "The determinant exists only for square
matrices (n == m).\n";
        cout << "Current size: " << n << "x" << m << " – not
square.\n";
    }
    else {
        int det = calcDeterminant(A);
        printResult(det);
    }
    break;

// Выход из программы
case 0:
    cout << "Exiting the program.\n";
    break;

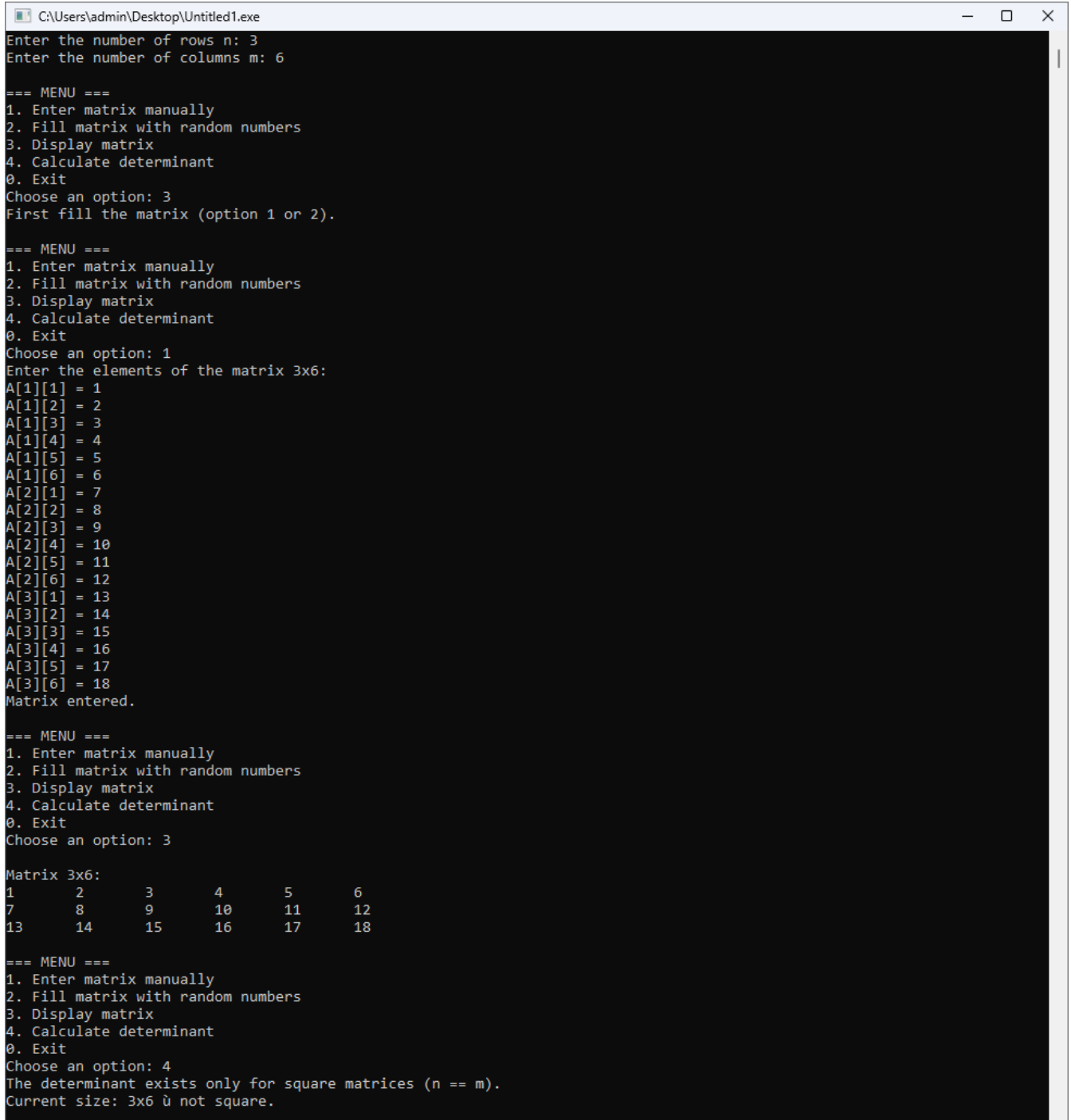
// Обработка неправильного ввода
default:
    cout << "Invalid choice. Try again.\n";
}

} while (choice != 0);

return 0;
}

```

4 Пример работы программы



```
C:\Users\admin\Desktop\Untitled1.exe
Enter the number of rows n: 3
Enter the number of columns m: 6

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 3
First fill the matrix (option 1 or 2).

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 1
Enter the elements of the matrix 3x6:
A[1][1] = 1
A[1][2] = 2
A[1][3] = 3
A[1][4] = 4
A[1][5] = 5
A[1][6] = 6
A[2][1] = 7
A[2][2] = 8
A[2][3] = 9
A[2][4] = 10
A[2][5] = 11
A[2][6] = 12
A[3][1] = 13
A[3][2] = 14
A[3][3] = 15
A[3][4] = 16
A[3][5] = 17
A[3][6] = 18
Matrix entered.

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 3

Matrix 3x6:
1      2      3      4      5      6
7      8      9      10     11     12
13     14     15     16     17     18

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 4
The determinant exists only for square matrices (n == m).
Current size: 3x6 ð not square.
```

Рисунок 1 – Ручной ввод матрицы


```
C:\Users\admin\Desktop\Untitled1.exe
Enter the number of rows n: 3
Enter the number of columns m: 3

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 2
Matrix filled with random numbers.

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 3

Matrix 3x3:
-9      9      0
-4      -8     -3
8       8     -4

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option: 4

Matrix determinant = -864

=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Display matrix
4. Calculate determinant
0. Exit
Choose an option:
```

Рисунок 2 – Случайное заполнение матрицы и подсчет определителя

5 Сравнение с кодом на Kotlin

5.1 Листинг программного кода

```
import kotlin.math.roundToInt
import java.util.Random

class DeterminantApp {

    private var n: Int = 0
    private var m: Int = 0
    private lateinit var matrix: Array<IntArray>
    private var filled = false

    fun run() {
        inputSize()
        matrix = Array(n) { IntArray(m) }

        var choice: Int
        do {
            showMenu()
            choice = readLine()!!.trim().toIntOrNull() ?: -1

            when (choice) {
                1 -> {
                    fillMatrix(matrix)
                    filled = true
                    println("Матрица введена.")
                }
                2 -> {
                    fillMatrixRandom(matrix)
                    filled = true
                }
                3 -> {
                    if (!filled) println("Сначала заполните матрицу (пункт 1 или 2).")
                    else printMatrix(matrix)
                }
                4 -> {
                    when {
                        !filled -> println("Сначала заполните матрицу (пункт 1 или 2).")
                        n != m -> println("Определитель существует только для квадратных матриц (n == m).\nТекущий размер: ${n}x${m} – не квадратная.")
                        else -> printResult(calcDeterminant(matrix))
                    }
                }
            }
        } while (choice != 4)
    }
}
```

```

        0 -> println("Выход из программы.")
        else -> println("Неверный выбор. Попробуйте снова.")
    }
} while (choice != 0)
}

fun inputSize() {
    print("Введите количество строк n: ")
    n = readLine()!!.trim().toIntOrNull() ?: 0
    while (n <= 0) {
        print("n должно быть больше 0. Введите снова: ")
        n = readLine()!!.trim().toIntOrNull() ?: 0
    }

    print("Введите количество столбцов m: ")
    m = readLine()!!.trim().toIntOrNull() ?: 0
    while (m <= 0) {
        print("m должно быть больше 0. Введите снова: ")
        m = readLine()!!.trim().toIntOrNull() ?: 0
    }
}

fun fillMatrix(a: Array<IntArray>) {
    println("Введите элементы матрицы ${n}x${m}:")
    for (i in 0 until n) {
        for (j in 0 until m) {
            print("A[${i + 1}][${j + 1}] = ")
            a[i][j] = readLine()!!.trim().toIntOrNull() ?: 0
        }
    }
}

fun fillMatrixRandom(a: Array<IntArray>) {
    val rng = Random()
    for (i in 0 until n) {
        for (j in 0 until m) {
            a[i][j] = rng.nextInt(19) - 9
        }
    }
    println("Матрица заполнена случайными числами.")
}

fun printMatrix(a: Array<IntArray>) {
    println("\nМатрица ${n}x${m}:")
    for (i in 0 until n) {
        println(a[i].joinToString("\t"))
    }
}

```

```

fun calcDeterminant(a: Array<IntArray>): Int {
    val b = Array(n) { i -> DoubleArray(n) { j -> a[i][j].toDouble() }
}

    var det = 1.0

    for (col in 0 until n) {
        val pivotRow = (col until n).firstOrNull { b[it][col] != 0.0 }

        if (pivotRow == null) return 0

        if (pivotRow != col) {
            val tmp = b[col]
            b[col] = b[pivotRow]
            b[pivotRow] = tmp
            det *= -1.0
        }

        det *= b[col][col]

        for (row in col + 1 until n) {
            val factor = b[row][col] / b[col][col]
            for (k in col until n) {
                b[row][k] -= factor * b[col][k]
            }
        }
    }

    return det.roundToInt()
}

fun printResult(det: Int) {
    println("\nОпределитель матрицы = $det")
}

fun showMenu() {
    println(
        """
=== МЕНЮ ===
1. Ввести матрицу вручную
2. Заполнить матрицу случайными числами
3. Вывести матрицу
4. Вычислить определитель
0. Выход
""").trimIndent()
    print("Выберите действие: ")
}

```

```

    }
}

fun main() {
    val app = DeterminantApp()
    app.run()
}

```

5.2 Пример работы программы

```

Enter number of rows n: 4
Enter number of columns m: 4
=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Print matrix
4. Calculate determinant
0. Exit
Choose an option: 2
Matrix filled with random numbers.
=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Print matrix
4. Calculate determinant
0. Exit
Choose an option: 3

Matrix 4x4:
-4  -1  6   0
2   -3  3  -3
5    4  -6  -7
-8   6  -5  -7
=== MENU ===
1. Enter matrix manually
2. Fill matrix with random numbers
3. Print matrix
4. Calculate determinant
0. Exit
Choose an option: 4

Matrix determinant = -1636

```

Рисунок 3 – Пример работы программы на Kotlin

5.3 Сравнение программы на Kotlin и C++

- Структура — практически одинакова: те же 7 функций с теми же именами. Главное отличие — у C++ есть `main()`, у Kotlin-скрипта код выполняется прямо «сверху вниз», без точки входа. `switch/case` → `when`, `do-while` — одинаковый
- Внешние зависимости — C++ подключает 5 заголовков вручную; Kotlin нужен только `import kotlin.math.roundToInt` и `import java.util.Random`, остальное доступно автоматически. Ключевое: Kotlin требует JVM, C++ — нет.
- Ввод/вывод — самое заметное различие. C++ использует `cin >>` (опасен при не-числе — программа зависает), Kotlin использует `readLine()!!.toIntOrNull()` — безопасный парсинг, никогда не падает. Вывод строки матрицы: C++ — ручной цикл, Kotlin — `joinToString("\n")` в одну строку.
- Типы данных — матрица `vector<vector<int>>` vs `Array<IntArray>`. Kotlin имеет встроенную null-safety (`toIntOrNull()` вместо потенциального краша).
- Безопасность — выход за границы массива в C++ — `undefined behavior` (молчаливый краш), в Kotlin — исключение с понятным сообщением.
- Производительность — алгоритм и сложность $O(n^3)$ идентичны. C++ быстрее стартует, Kotlin потребляет ~50–200 МБ лишней памяти на JVM.
- Читаемость — Kotlin компактнее примерно на 30% (115 vs 163 строки) за счёт шаблонных строк, `when`, `firstOrNull`, `joinToString`.
- Переносимость — C++ бинарник зависит от ОС и архитектуры; один `.kts`-файл Kotlin запускается везде, где есть JVM.

6 Расчет теоретических пространственной и временной сложностей алгоритма

6.1 Пространственная сложность

Программа использует следующие данные:

- две глобальные переменные `int n` и `int m`;
- двумерный массив `A` размером $n \times m$ типа `int` (основное хранилище матрицы);
- внутри `calcDeterminant`: временный массив `B` размером $n \times n$ типа `double`, а также скалярные переменные `det`, `factor`, `pivotRow`, `col`, `row`, `k` — $O(1)$ штук;
- в остальных функциях: только скалярные переменные `i`, `j`, `choice`, `filled` — $O(1)$ штук.

Пространственная сложность определяется формулой: $V = 2 \cdot C_{\text{int}} + n \cdot m \cdot C_{\text{int}} + n^2 \cdot C_{\text{double}} + C_{\text{const}}$

При $n = m$ квадратная матрица даёт $V = O(n^2)$. Таким образом, пространственная сложность всей программы: $V(n, m) = O(n \cdot \max(n, m))$.

6.2 Временная сложность функций

Обозначения: K_i — константа, характеризующая время выполнения операции на строке i .

Функция	Формула t	$T(n, m)$
<code>inputSize()</code>	$t = K_{12} + K_{13} + K_{18} + K_{19}$	$O(1)$
<code>fillMatrix()</code>	$t = n \cdot m \cdot K_{32}$	$O(n \cdot m)$
<code>fillMatrixRandom()</code>	$t = K_{39} + n \cdot m \cdot K_{42}$	$O(n \cdot m)$
<code>printMatrix()</code>	$t = n \cdot m \cdot K_{53}$	$O(n \cdot m)$
<code>calcDeterminant()</code>	$t = n^2 \cdot K_{\text{copy}} + n \cdot (n \cdot K_{\text{pivot}} + n^2 \cdot K_{\text{elim}})$	$O(n^3)$
<code>printResult()</code>	$t = K_{103}$	$O(1)$
<code>showMenu()</code>	$t = 7 \cdot K_{\text{cout}}$	$O(1)$

6.3 Подробный расчёт для calcDeterminant

Функция состоит из трёх частей:

1. Копирование $A \rightarrow B$: двойной цикл $n \times n \rightarrow t_copy = n^2 \cdot K_copy$
2. Поиск опорного элемента (pivotRow): внутри внешнего цикла по col (n итераций) внутренний цикл от col до n-1, в худшем случае n итераций $\rightarrow$ суммарно $O(n^2)$
3. Обнуление элементов ниже опорного: для каждого col выполняется (n-col-1) итераций по строкам, каждая содержит цикл по k из (n-col) элементов:

$$t_elim = \sum_{col=0..n-1} (n-col-1) \cdot (n-col) \approx n^3/3$$

Итоговая формула:

$$t_calcDet = n^2 \cdot K_copy + n^2 \cdot K_pivot + n^3/3 \cdot K_elim$$

$$T_calcDet(n) = O(\max(n^2, n^2, n^3)) = O(n^3)$$

6.4 Временная сложность всей программы

$$t_alg = t_inputSize + t_fillMatrix \text{ (или } t_fillMatrixRandom) + t_printMatrix + t_calcDeterminant + t_printResult$$

$$T_alg(n, m) = O(\max(O(1), O(n \cdot m), O(n \cdot m), O(n^3), O(1)))$$

$$\text{При } n = m: T_alg(n) = O(\max(n^2, n^3)) = O(n^3).$$

$$\text{При } n \neq m \text{ вычисление определителя недоступно, тогда } T_alg(n, m) = O(n \cdot m).$$

7 Выводы по работе

В ходе лабораторной работы разработана программа на C++ для вычисления определителя матрицы методом Гаусса.

Особенности реализации:

1. Матрица хранится как двумерный массив int размером $n \times m$.
Поскольку определитель существует только для квадратных матриц, пункт 4 меню доступен лишь при $n = m$.

2. Предусмотрено два способа заполнения матрицы: вручную (`fillMatrix`) и случайными числами от -9 до 9 (`fillMatrixRandom`) — оба вызываются через единое меню.
3. Внутри `calcDeterminant` элементы копируются в массив `double`, так как метод Гаусса использует деление и целочисленная арифметика привела бы к потере точности. Результат округляется и возвращается как `int`, поскольку определитель целочисленной матрицы является целым числом.
4. Каждое действие (ввод размеров, ввод вручную, случайное заполнение, вывод, вычисление, вывод результата, меню) оформлено отдельной функцией.

Анализ сложности показал:

- пространственная сложность: $O(n \cdot \max(n, m))$ — при квадратной матрице $O(n^2)$;
- временная сложность: $O(n^3)$ при $n = m$ — определяется тройным вложенным циклом в методе Гаусса; $O(n \cdot m)$ при $n \neq m$ — определяется заполнением и выводом матрицы.

Алгоритм Гаусса с полиномиальной сложностью $O(n^3)$ значительно эффективнее разложения по строке/столбцу, имеющего факториальную сложность $O(n!)$.